

第21回 Canvas の基本

実習時のフォルダ構成

第 20 回と同じ形式で、Web_名前 の中に 21 フォルダを作る。21 の中に sample と exercise を作り、次のファイルを用意する。

```
Web_名前
├── 21
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、すべて sample/main.js に書く）
- exercise：演習で作るファイル（演習 1～2、発展）

sample/index.html と exercise/index.html には、どちらも次の内容を書く。今回から、<body> の中に <canvas> を書く。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>Canvasの基本</title>
</head>
<body>
  <canvas id="game" width="400" height="500"></canvas>
  <script src="main.js"></script>
</body>
</html>
```

- <canvas>：JavaScript で絵を描くための「キャンバス（画用紙）」を、ページに置くタグ。
- width="400" height="500"：キャンバスの幅と高さ（単位はピクセル）。
- id="game"：JavaScript からこのキャンバスを取り出すための名前。

講義内容

以降の講義内容のコードは、sample/main.js に、上から順に書き足していく。書いたら sample/index.html をブラウザで開き、画面とコンソールで結果を確認する。

1. 前回の復習

- 関数：処理をまとめて名前を付けたもの。function 名前(引数) { } で作り、名前(値); で呼び出す。

- `document.getElementById("id名")` で HTML の要素を取り出し、`.textContent` で中の文字を書き換えられる。

今回は、関係のある値をまとめる「オブジェクト」と、Canvas（キャンバス）に図形や文字を描く方法を学ぶ。

2. オブジェクト

自機の位置（x, y）や大きさ（幅, 高さ）のように、1 つのものに関する値をまとめて 1 つにしたものを「オブジェクト」という。`{ }` の中に、名前: 値 を `,` で区切って並べる。

```
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
console.log(player.x);           // 184 (値の読み取り)
console.log(player.w);           // 32
player.x = 100;                  // 値の変更
console.log(player.x);           // 100
```

名前（プロパティ）	値	意味
x	184	横の位置
y	440	縦の位置
w	32	幅（width）
h	32	高さ（height）
speed	5	速さ（次回から使う）

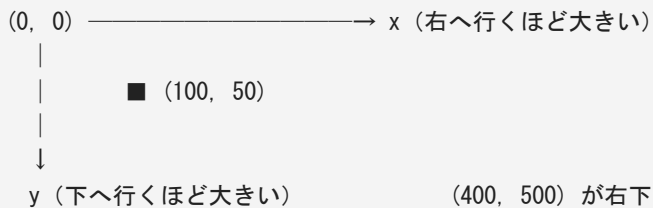
- オブジェクトの中の 1 つ 1 つの値を「プロパティ」という。
- `オブジェクト.プロパティ名` で値を取り出す（例：`player.x`）。`player.x = 100;` のように、値を書き換えることもできる。
- 配列は「番号」で取り出し、オブジェクトは「名前」で取り出す。
- 1 人の自機に関する値を、`player` という 1 つの名前でまとめて扱える。

3. Canvas を使う準備と座標

```
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");
```

- 1 行目は、第 20 回と同じ `getElementById`。id が `game` の `<canvas>` を取り出す。
- 2 行目の `canvas.getContext("2d")` が、今回の新しい書き方。キャンバスに絵を描くための道具（ペンや絵の具のようなもの）を取り出す。`ctx` は context（コンテキスト）の略。
- `getElementById` の `"game"` と、`<canvas>` の `id="game"` は、同じ名前にする。違うと、エラーになり何も描かれない。

キャンバスの位置は、左上を (0, 0) として、x 座標（横）と y 座標（縦）で表す。



- x は右へ行くほど大きく、y は下へ行くほど大きい（数学のグラフとは上下が逆）。
- キャンバスの幅は `canvas.width` (400)、高さは `canvas.height` (500) で取り出せる。

4. 四角形を描く

```
ctx.fillStyle = "#000022";  
ctx.fillRect(0, 0, canvas.width, canvas.height);
```

```
ctx.fillStyle = "#ff4444";  
ctx.fillRect(50, 150, 80, 40);
```

- `ctx.fillStyle = "色";` : これから塗る色を決める。色は、CSS と同じ書き方 (`#rrggbb` など)。
- `ctx.fillRect(x, y, 幅, 高さ);` : 左上が (x, y) の位置に、塗りつぶした四角形を描く。
- 1 つ目は、キャンバス全体 ((0, 0) から幅 400・高さ 500) を塗って、背景にしている。
- 後から描いたものが、上に重なる。そのため、背景は最初に描く。

5. 文字を描く

Canvas には、文字も描ける。

```
ctx.fillStyle = "#ffffff";  
ctx.font = "20px sans-serif";  
ctx.textAlign = "center";  
ctx.fillText("SHOOTING GAME", canvas.width / 2, 250);
```

- `ctx.font = "大きさ フォント";` : 文字の大きさとフォントを決める。
- `ctx.fillText("文字", x, y);` : (x, y) の位置に文字を描く。y は、文字の**ベースライン**（ほぼ文字の下端。g や y などは、少し下にはみ出す）の位置。
- `ctx.textAlign` : "left" にすると x が文字の左端、"center" にすると x が文字の中央になる。
`canvas.width / 2` (200) と "center" で、文字を横の真ん中に置ける。
- 最後に作るシューティングゲームでは、SCORE（得点）や LIVES（残機）は、Canvas には描かず、第 20 回と同じく HTML の表示欄に出す。Canvas には、自機・弾・敵などの動くものを描く。

6. for 文で並べて描く

第 19 回の for 文の `i` を座標の計算に使うと、同じ形を並べて描ける。

```
ctx.fillStyle = "#ff4444";
for (let i = 0; i < 5; i++) {
  ctx.fillRect(40 + i * 70, 70, 32, 32);
}
```

i	x 座標 $40 + i * 70$
0	40
1	110
2	180
3	250
4	320

7. オブジェクトと関数で自機を描く

講義 2 で作った `player` を使い、自機を描く処理を、第 20 回で学んだ関数にまとめる。

```
function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

drawPlayer();
```

- `player` の `x`・`y`・`w`・`h` が、`fillRect` の `x`・`y`・幅・高さに入る。
- 講義 2 で `player.x` を 100 に変えたので、自機は `x` が 100 の位置に描かれる。`player.x` の値を変えてから `drawPlayer()` を呼び出すと、違う位置に描ける。
- 次回から、この `player` と `drawPlayer` を使って、自機を動かす。`speed`（速さ）も、次回から使う。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて確認する。

演習 1 背景と敵を描く

- Canvas を使う準備の 2 行を書く。
- キャンバス全体を、好きな色で塗って背景にする。
- for 文を使って、上の方に敵（32 × 32 の四角形）を横に 6 つ並べて描く。色は自分で決める。

演習 2 オブジェクトと関数で自機を描く

- 自機をオブジェクトで作る（`x`・`y`・`w`・`h`・`speed`。値は自分で決める）。

- 自機を描く関数 `drawPlayer` を作り、呼び出して、キャンバスの下の方に描く。
- `player.x` の値を変えて、描かれる位置が変わることを確認する。

発展（任意）

- `fillText` で、キャンバスの中央にタイトルの文字（例：「SHOOTING GAME」）を描く。
- もう 1 つ `for` 文を書いて、敵の下段に、別の色で 6 つ並べて描く。
- 自機の周りに四角形を描き足して、翼や先端のある形にする。位置は `player.x` ・ `player.y` をもとに計算する（例：`player.x - 12`）。
- `for` 文と乱数を使って、小さな白い四角形（ 2×2 ）を 30 個、ランダムな位置に描いて、星空にする。